

Chip in, big win

UVM Sequence 常考题 / 真题汇总

Part 1 / 3 · 题目 1 ~ 8

SystemVerilog · UVM 1.2 / IEEE 1800.2

本册涵盖题目 1-8：Ping/ACK 握手状态机（题1）、Alert→ESC 升级 FSM（题2）、AXI4 4KB 边界约束（题3）、Cache LRU 驱逐（题4）、APB PSLVERR 错误注入（题5）、AHB WRAP4 地址回绕（题6）、SPI 四模式覆盖（题7）、VSeq 串行配置+并行数据屏障（题8）。

题目 1　Alert/Escalation Ping/ACK 握手状态机 + 超时重试

难度：中级
标签：start_item / finish_item / get_response / inline constraint / retry loop

【题目背景】

OpenTitan 平台的 Alert/Escalation 总线使用 Ping-Pong 握手机制来检测 总线完整性。Alert Sender 定期向 Alert Receiver 发出 PING_REQ， Receiver 须在规定时间内回复有效 ACK（ack==1）。 若连续多次未收到 ACK，则触发升级（Escalation）处理。

【完整要求】

- 1. 序列循环发送 num_pings 次 Ping 请求；
- 2. 每次 Ping 使用 inline constraint 约束 alert_type == PING_REQ；
- 3. 通过 get_response(rsp) 获取 driver 回填的 ack 字段；
- 4. 若 ack==0，则重试，最多重试 max_retry 次；
- 5. max_retry 次后仍无 ACK，调用 `uvm_fatal 报错；
- 6. 每次重试需打印 `uvm_warning 提示当前是第几次尝试；
- 7. 成功收到 ACK 后打印 `uvm_info 记录成功轮次。

【实现要点】

> - 用 while(!got_ack && retry < max_retry) 而非 repeat 实现有条件重试；
> - inline constraint: req.randomize() with { alert_type == PING_REQ; }
> - 不得在重试循环外部提前调用 item_done，避免 sequencer 状态机错乱。

【完整答案代码（含全部类定义）】

```
class alert_esc_seq_item extends uvm_sequence_item;
    typedef enum bit [1:0] {PING_REQ, ALERT_REQ, ESC_REQ, NOP} alert_type_e;

    rand alert_type_e alert_type;
    rand bit          ack;
    rand bit [7:0]    payload;

    `uvm_object_utils_begin(alert_esc_seq_item)
        `uvm_field_enum(alert_type_e, alert_type, UVM_ALL_ON)
        `uvm_field_int(ack,     UVM_ALL_ON)
        `uvm_field_int(payload, UVM_ALL_ON)
    `uvm_object_utils_end

    function new(string name = "alert_esc_seq_item");
        super.new(name);
    endfunction
endclass

class alert_esc_ping_sequence extends uvm_sequence#(alert_esc_seq_item);
    `uvm_object_utils(alert_esc_ping_sequence)

    int unsigned num_pings;
    int unsigned max_retry;

    function new(string name = "alert_esc_ping_sequence");
        super.new(name);
        num_pings = 10;
        max_retry = 3;
    endfunction

    task body();
        alert_esc_seq_item req, rsp;
        repeat (num_pings) begin
            int retry  = 0;
            bit got_ack = 0;
            // 考点: 基于响应驱动重试，不是简单 repeat
            while (!got_ack && retry < max_retry) begin
                req = alert_esc_seq_item::type_id::create("ping_item");
                start_item(req);
                if (!req.randomize() with {
                    alert_type == alert_esc_seq_item::PING_REQ;
                }) `uvm_fatal("RAND_FAIL", "Ping randomization failed")
                finish_item(req);
                get_response(rsp);
                got_ack = rsp.ack;
                retry++;
                if (!got_ack)
                    `uvm_warning("PING_SEQ",
                        $sformatf("No ACK on attempt %0d, retrying...", retry))
            end
            if (!got_ack)
                `uvm_fatal("PING_SEQ",
                    $sformatf("Ping failed after %0d retries", max_retry))
            else
                `uvm_info("PING_SEQ",
                    $sformatf("ACK received on attempt %0d", retry), UVM_MEDIUM)
        end
    endtask
endclass
```

题目 2　Alert → ESC 升级序列状态机（FSM inline constraint）

难度：中级
标签：FSM / inline constraint / ALERT_REQ / ESC_REQ / 状态驱动序列

【题目背景】

Alert/Escalation 协议定义了合法的命令升级流程： ALERT_REQ（第一级告警）→ 若无 ACK → ESC_REQ（升级）→ NOP（结束） 序列必须严格按照此状态机顺序产生事务，不能随机打乱顺序。

【完整要求】

- 1. 序列内部维护一个三状态 FSM：S_ALERT → S_ESCALATE → S_NOP；
- 2. 初始状态为 S_ALERT，每轮（scenario）从头开始；
- 3. S_ALERT 状态：发送 ALERT_REQ；
- rsp.ack==1 → 转移到 S_NOP（对方已确认，无需升级）；
- rsp.ack==0 → 转移到 S_ESCALATE（升级）；
- 4. S_ESCALATE 状态：发送 ESC_REQ，然后无条件转移到 S_NOP；
- 5. S_NOP 状态：发送 NOP，结束本轮 scenario；
- 6. 重复 num_scenarios 轮，覆盖"直接确认"和"升级后结束"两条路径。

【实现要点】

> - case(state) 配合 inline constraint 切换 alert_type，实现类型安全；
> - 每轮内层用 forever + break(S_NOP) 驱动状态机循环；
> - 外层 repeat(num_scenarios) 驱动多轮覆盖。

【完整答案代码（含全部类定义）】

```
class alert_escalation_fsm_sequence extends uvm_sequence#(alert_esc_seq_item);
    `uvm_object_utils(alert_escalation_fsm_sequence)

    int unsigned num_scenarios;

    function new(string name = "alert_escalation_fsm_sequence");
        super.new(name);
        num_scenarios = 5;
    endfunction

    task body();
        alert_esc_seq_item req, rsp;
        typedef enum { S_ALERT, S_ESCALATE, S_NOP } fsm_state_e;
        fsm_state_e state;

        repeat (num_scenarios) begin
            state = S_ALERT;
            forever begin
                req = alert_esc_seq_item::type_id::create("esc_item");
                start_item(req);
                // 考点: inline constraint 与 FSM 状态绑定
                case (state)
                    S_ALERT: begin
                        if (!req.randomize() with {
                            alert_type == alert_esc_seq_item::ALERT_REQ;
                        }) `uvm_fatal("RAND_FAIL","ALERT rand failed")
                    end
                    S_ESCALATE: begin
                        if (!req.randomize() with {
                            alert_type == alert_esc_seq_item::ESC_REQ;
                        }) `uvm_fatal("RAND_FAIL","ESC rand failed")
                    end
                    S_NOP: begin
                        if (!req.randomize() with {
                            alert_type == alert_esc_seq_item::NOP;
                        }) `uvm_fatal("RAND_FAIL","NOP rand failed")
                    end
                endcase
                finish_item(req);
                get_response(rsp);
                // 状态转移: 无 ACK → 升级; 有 ACK → NOP
                case (state)
                    S_ALERT:    state = rsp.ack ? S_NOP : S_ESCALATE;
                    S_ESCALATE: state = S_NOP;
                    S_NOP: begin
                        `uvm_info("ESC_SEQ","Scenario done",UVM_MEDIUM)
                        break;
                    end
                endcase
            end
        end
    endtask
endclass
```

题目 3　AXI4 地址对齐约束 + 4KB 边界不越界检查

难度：中级
标签：AXI4 / 4KB boundary / constraint / addr_align / 参考模型断言

【题目背景】

AXI4 协议（AMBA AXI and ACE Protocol Specification）第 A3.4.1 节规定： 任何突发传输不得跨越 4KB 地址边界（4096 字节对齐边界）。 这是 AXI 互联实现的基本假设，违反此规则将导致地址译码错误。

【完整要求】

(addr[11:0] + (1<<size) × (len+1)) <= 4096； 若 end_offset > 4096 则 `uvm_fatal（constraint 失效的二次保障）；

- 1. 在 transaction constraint 中用 no_4k_cross_c 约束：
- 2. size 只允许 0~3（对应 1/2/4/8 字节）；
- 3. addr 按最大 size（8B）对齐：addr[2:0] == 3'b000；
- 4. sequence body() 中对每笔事务调用参考模型独立计算 end_offset，
- 5. 生成 num_trans 笔随机读写事务，打印 kind/addr/len/size 信息。

【实现要点】

> - constraint 中 (1 << size) 是 SystemVerilog 位宽表达式，需注意宽度匹配；
> - sequence 层二次断言是面试高频考点：constraint 是随机化指导，
> - 不等于硬件行为保证，必须显式检查。

【完整答案代码（含全部类定义）】

```
class axi_transaction extends uvm_sequence_item;
    typedef enum bit { AXI_WRITE, AXI_READ } axi_kind_e;

    rand axi_kind_e   kind;
    rand bit [31:0]   addr;
    rand bit [7:0]    len;    // burst length - 1
    rand bit [2:0]    size;   // bytes per beat = 2^size
    rand bit [63:0]   data;

    `uvm_object_utils_begin(axi_transaction)
        `uvm_field_enum(axi_kind_e, kind, UVM_ALL_ON)
        `uvm_field_int(addr, UVM_ALL_ON)
        `uvm_field_int(len,  UVM_ALL_ON)
        `uvm_field_int(size, UVM_ALL_ON)
        `uvm_field_int(data, UVM_ALL_ON)
    `uvm_object_utils_end

    constraint legal_size_c  { size inside {3'b000, 3'b001, 3'b010, 3'b011}; }
    constraint addr_align_c  { addr[2:0] == 3'b000; }
    // 考点: 4KB 边界约束
    constraint no_4k_cross_c {
        (addr[11:0] + ((1 << size) * (len + 1))) <= 4096;
    }

    function new(string name = "axi_transaction");
        super.new(name);
    endfunction
endclass

class axi_4k_boundary_sequence extends uvm_sequence#(axi_transaction);
    `uvm_object_utils(axi_4k_boundary_sequence)

    int unsigned num_trans;

    function new(string name = "axi_4k_boundary_sequence");
        super.new(name);
        num_trans = 20;
    endfunction

    task body();
        axi_transaction req, rsp;
        repeat (num_trans) begin
            req = axi_transaction::type_id::create("axi_txn");
            start_item(req);
            if (!req.randomize())
                `uvm_fatal("RAND_FAIL","AXI 4KB rand failed")
            // 考点: 序列层参考模型二次断言
            begin
                int beat_bytes = 1 << int'(req.size);
                int end_offset = req.addr[11:0] + beat_bytes * (req.len + 1);
                if (end_offset > 4096)
                    `uvm_fatal("AXI_4KB",
                        $sformatf("4KB violation! addr=0x%08h len=%0d size=%0d",
                            req.addr, req.len, req.size))
            end
            finish_item(req);
            get_response(rsp);
            `uvm_info("AXI_SEQ",
                $sformatf("%s addr=0x%08h len=%0d size=%0d",
                    req.kind.name(), req.addr, req.len+1, req.size), UVM_MEDIUM)
        end
    endtask
endclass
```

题目 4　Cache LRU 驱逐验证（组相联几何计算 + miss 断言）

难度：高级
标签：Cache / LRU / set-associative / eviction / cold miss / hit/miss

【题目背景】

假设目标 SoC 的 L1 数据 Cache 参数如下： - 容量：64 KB - 组相联度：4 路（4-way set-associative） - Cache 行大小：64 字节 - 替换策略：LRU（Least Recently Used） - 主存地址空间：4 GB（32-bit 地址） 需要通过 UVM 序列精确构造地址，验证 Cache 控制器在驱逐时 确实淘汰的是最久未使用的 Cache 行。

【完整要求】

（SETS、OFFSET_BITS、INDEX_BITS），并注释推导过程； Step A：依次访问 tag=0..WAYS-1（填满该 set，期望全部 cold miss）； Step B：访问 tag=WAYS（第 WAYS+1 个不同 tag），触发 LRU 替换； Step C：回访 tag=0 的地址（LRU 最老），断言 hit==0（已被驱逐）；

- 1. 在 sequence 中用 localparam 推导所有 Cache 几何参数
- 2. 实现 make_addr(tag, set_idx) 函数，按位域拼接地址；
- 3. 对每个 set（共 256 个），执行以下步骤：
- 4. 步骤 A 中若 hit==1 则打印 `uvm_warning（可能是脏数据残留）；
- 5. 步骤 C 中若 hit==1 则打印 `uvm_error（LRU 驱逐失败）。

【实现要点】

> - tag=0 是最早访问的，按 LRU 策略是第一个被替换的；
> - step B 的访问期望 miss（新 tag，触发替换），若 hit 则报 error；
> - 地址构造：make_addr(t, s) = (t << 14) | (s << 6)。

【完整答案代码（含全部类定义）】

```
class cache_access_item extends uvm_sequence_item;
    rand bit [31:0] addr;
    bit             hit;    // driver 回填: 1=hit, 0=miss

    `uvm_object_utils_begin(cache_access_item)
        `uvm_field_int(addr, UVM_ALL_ON)
        `uvm_field_int(hit,  UVM_ALL_ON)
    `uvm_object_utils_end

    function new(string name = "cache_access_item");
        super.new(name);
    endfunction
endclass

class cache_eviction_sequence extends uvm_sequence#(cache_access_item);
    `uvm_object_utils(cache_eviction_sequence)

    // 考点: 参数推导
    localparam int WAYS        = 4;
    localparam int LINE_BYTES  = 64;
    localparam int CACHE_BYTES = 64 * 1024;
    localparam int SETS        = CACHE_BYTES / (WAYS * LINE_BYTES); // 256
    localparam int OFFSET_BITS = 6;
    localparam int INDEX_BITS  = 8;

    function new(string name = "cache_eviction_sequence");
        super.new(name);
    endfunction

    function bit [31:0] make_addr(int tag, int set_idx);
        return (tag << (INDEX_BITS + OFFSET_BITS)) | (set_idx << OFFSET_BITS);
    endfunction

    task body();
        cache_access_item req, rsp;
        bit [31:0] lru_addr;

        for (int s = 0; s < SETS; s++) begin
            // 填满: 冷缺失, tag 0..WAYS-1
            for (int t = 0; t < WAYS; t++) begin
                req = cache_access_item::type_id::create("fill");
                start_item(req);
                req.addr = make_addr(t, s);
                finish_item(req);
                get_response(rsp);
                if (rsp.hit)
                    `uvm_warning("CACHE_SEQ",
                        $sformatf("Expected cold miss: set=%0d tag=%0d",s,t))
            end
            lru_addr = make_addr(0, s);   // tag=0 是 LRU

            // 触发替换
            req = cache_access_item::type_id::create("evict_trigger");
            start_item(req);
            req.addr = make_addr(WAYS, s);
            finish_item(req);
            get_response(rsp);
            if (rsp.hit)
                `uvm_error("CACHE_SEQ",
                    $sformatf("Should miss on eviction trigger: set=%0d",s))

            // 考点: 验证 LRU 已被驱逐 (hit 必须为 0)
            req = cache_access_item::type_id::create("lru_verify");
            start_item(req);
            req.addr = lru_addr;
            finish_item(req);
            get_response(rsp);
            if (rsp.hit)
                `uvm_error("CACHE_SEQ",
                    $sformatf("LRU eviction failed: set=%0d addr=0x%08h still hit",
                        s, lru_addr))
            else
```

```
                `uvm_info("CACHE_SEQ",
                    $sformatf("LRU eviction OK: set=%0d",s), UVM_LOW)
        end
    endtask
endclass
```

题目 5　APB PSLVERR 错误注入与检测 + 错误率统计断言

难度：中级
标签：APB / PSLVERR / error injection / address decode / 统计断言

【题目背景】

APB（Advanced Peripheral Bus）协议中，slave 对不支持的地址区间 须在 ACCESS 阶段末尾拉高 PSLVERR 信号，表示传输出错。 验证 APB slave 的地址解码逻辑时，需要主动向非法地址发起访问， 并验证 PSLVERR 被正确置位，且合法地址访问不误触发 PSLVERR。

【完整要求】

不足则报 `uvm_error 并打印期望值与实际值。

- 1. Phase 1（合法地址，num_legal=16 次）：
- 约束 paddr 在 LEGAL_BASE ~ LEGAL_TOP 区间内；
- 每笔事务收到响应后检查 pslverr，若为 1 则报 `uvm_error；
- 2. Phase 2（非法地址，num_illegal=8 次）：
- 约束 paddr 在 ILLEGAL_BASE ~ ILLEGAL_TOP 区间内；
- 方向固定为 APB_WRITE；
- 每笔事务收到响应后检查 pslverr，若为 0 则报 `uvm_error；
- 统计成功触发 PSLVERR 的次数 slverr_cnt；
- 3. 最终统计断言：slverr_cnt 必须等于 num_illegal（100% 命中率）；

【实现要点】

> - 两种地址区间用 localparam 定义，constraint 中用 inside 集合选择；
> - 统计断言是本题核心：单笔检查 + 最终汇总，双重保障。

【完整答案代码（含全部类定义）】

```
class apb_transaction extends uvm_sequence_item;
    typedef enum bit { APB_WRITE, APB_READ } apb_dir_e;

    rand apb_dir_e  dir;
    rand bit [31:0] paddr;
    rand bit [31:0] pwdata;
    bit      [31:0] prdata;
    bit             pslverr;   // driver 回填

    `uvm_object_utils_begin(apb_transaction)
        `uvm_field_enum(apb_dir_e, dir,  UVM_ALL_ON)
        `uvm_field_int(paddr,            UVM_ALL_ON)
        `uvm_field_int(pwdata,           UVM_ALL_ON)
        `uvm_field_int(prdata,           UVM_ALL_ON)
        `uvm_field_int(pslverr,          UVM_ALL_ON)
    `uvm_object_utils_end

    constraint addr_align_c { paddr[1:0] == 2'b00; }

    function new(string name = "apb_transaction");
        super.new(name);
    endfunction
endclass

class apb_error_injection_sequence extends uvm_sequence#(apb_transaction);
    `uvm_object_utils(apb_error_injection_sequence)

    localparam bit [31:0] LEGAL_BASE   = 32'h0000_0000;
    localparam bit [31:0] LEGAL_TOP    = 32'h0000_0FFC;
    localparam bit [31:0] ILLEGAL_BASE = 32'h0000_F000;
    localparam bit [31:0] ILLEGAL_TOP  = 32'h0000_FFFC;

    int unsigned num_legal;
    int unsigned num_illegal;

    function new(string name = "apb_error_injection_sequence");
        super.new(name);
        num_legal   = 16;
        num_illegal = 8;
    endfunction

    task body();
        apb_transaction req, rsp;
        int slverr_cnt = 0;

        // Phase 1: 合法地址，不应有 PSLVERR
        repeat (num_legal) begin
            req = apb_transaction::type_id::create("apb_legal");
            start_item(req);
            if (!req.randomize() with {
                paddr inside {[LEGAL_BASE:LEGAL_TOP]};
                paddr[1:0] == 2'b00;
            }) `uvm_fatal("RAND_FAIL","APB legal rand failed")
            finish_item(req);
            get_response(rsp);
            if (rsp.pslverr)
                `uvm_error("APB_ERR",
                    $sformatf("Unexpected PSLVERR on legal addr=0x%08h",rsp.paddr))
        end

        // Phase 2: 非法地址，必须收到 PSLVERR=1
        repeat (num_illegal) begin
            req = apb_transaction::type_id::create("apb_illegal");
            start_item(req);
            if (!req.randomize() with {
                paddr inside {[ILLEGAL_BASE:ILLEGAL_TOP]};
                paddr[1:0] == 2'b00;
                dir == apb_transaction::APB_WRITE;
            }) `uvm_fatal("RAND_FAIL","APB illegal rand failed")
            finish_item(req);
            get_response(rsp);
            if (rsp.pslverr) slverr_cnt++;
```

```
            else
                `uvm_error("APB_ERR",
                    $sformatf("Missing PSLVERR on illegal addr=0x%08h",rsp.paddr))
        end

        // 考点: 统计断言
        if (slverr_cnt != num_illegal)
            `uvm_error("APB_ERR",
                $sformatf("PSLVERR count mismatch: exp=%0d got=%0d",
                    num_illegal, slverr_cnt))
        else
            `uvm_info("APB_ERR",
                $sformatf("Error injection passed: %0d/%0d PSLVERR correct",
                    slverr_cnt, num_illegal), UVM_LOW)
    endtask
endclass
```

题目 6　AHB WRAP4 地址回绕参考模型逐拍比对

难度：高级
标签：AHB / WRAP4 / burst / address wrap / 参考模型 / HTRANS / NONSEQ / SEQ

【题目背景】

AHB-Lite 的 WRAP（环绕型）突发要求地址在固定边界内循环回绕， 而非线性递增。对于 WRAP4（4 拍突发），wrap boundary = 4 × HSIZE 字节。 例如 HSIZE=WORD（4B），wrap boundary=16B：首地址 0x10 的 WRAP4 突发 地址序列为 0x10 → 0x14 → 0x18 → 0x1C → 0x10（回绕到起点）。 DUT 的地址生成逻辑容易在边界处出错，必须逐拍验证。

【完整要求】

wrap_bnd = 4 × (1<<hsize)； mask = wrap_bnd - 1； result = (base & ~mask) | ((base + beat × (1<<hsize)) & mask)； 不一致则 `uvm_error；

- 1. 在 sequence 中实现 wrap4_addr(base, beat, hsize) 参考模型函数：
- 2. 每次突发随机选取 WORD 对齐基地址（base[1:0]==2'b00）；
- 3. 发送 NONSEQ 首拍（htrans=NONSEQ, hburst=WRAP4, hsize=WORD）；
- 4. 发送 SEQ 拍 1~3（htrans=SEQ），haddr 由参考模型计算设定；
- 5. 每拍收到 get_response 后，将 rsp.haddr_actual 与参考模型结果比对，
- 6. 重复 num_bursts 次突发。

【实现要点】

> - haddr 字段在 SEQ 拍中由序列层强制指定（不随机），模拟 master 行为；
> - driver 回填 haddr_actual 表示 DUT 输出给总线的地址，二者必须一致；
> - WRAP4 在 beat=3 后的下一拍会回到基地址，此处不发送第 5 拍。

【完整答案代码（含全部类定义）】

```
class ahb_transaction extends uvm_sequence_item;
    typedef enum bit [1:0] {
        AHB_IDLE   = 2'b00, AHB_BUSY = 2'b01,
        AHB_NONSEQ = 2'b10, AHB_SEQ  = 2'b11
    } htrans_e;

    typedef enum bit [2:0] {
        SINGLE=3'b000, INCR  =3'b001,
        WRAP4 =3'b010, INCR4 =3'b011,
        WRAP8 =3'b100, INCR8 =3'b101,
        WRAP16=3'b110, INCR16=3'b111
    } hburst_e;

    rand htrans_e   htrans;
    rand hburst_e   hburst;
    rand bit        hwrite;
    rand bit [31:0] haddr;
    rand bit [31:0] hwdata;
    rand bit [2:0]  hsize;
         bit [31:0] hrdata;
         bit [31:0] haddr_actual;   // driver 回填 DUT 实际地址

    `uvm_object_utils_begin(ahb_transaction)
        `uvm_field_enum(htrans_e, htrans,      UVM_ALL_ON)
        `uvm_field_enum(hburst_e, hburst,      UVM_ALL_ON)
        `uvm_field_int(hwrite,                 UVM_ALL_ON)
        `uvm_field_int(haddr,                  UVM_ALL_ON)
        `uvm_field_int(hwdata,                 UVM_ALL_ON)
        `uvm_field_int(hsize,                  UVM_ALL_ON)
        `uvm_field_int(hrdata,                 UVM_ALL_ON)
        `uvm_field_int(haddr_actual,           UVM_ALL_ON)
    `uvm_object_utils_end

    function new(string name = "ahb_transaction");
        super.new(name);
    endfunction
endclass

class ahb_wrap4_sequence extends uvm_sequence#(ahb_transaction);
    `uvm_object_utils(ahb_wrap4_sequence)

    int unsigned num_bursts;

    function new(string name = "ahb_wrap4_sequence");
        super.new(name);
        num_bursts = 8;
    endfunction

    // 考点: WRAP4 地址参考模型
    function bit [31:0] wrap4_addr(bit [31:0] base, int beat, int hsize);
        int beat_bytes = 1 << hsize;
        int wrap_bnd   = 4 * beat_bytes;
        bit [31:0] mask = wrap_bnd - 1;
        return (base & ~mask) | ((base + beat * beat_bytes) & mask);
    endfunction

    task body();
        ahb_transaction req, rsp;
        bit [31:0] base_addr;

        repeat (num_bursts) begin
            void'(std::randomize(base_addr) with {
                base_addr[1:0] == 2'b00;   // WORD 对齐
                base_addr <= 32'hFFFF_FFF0;
            });

            // NONSEQ 首拍
            req = ahb_transaction::type_id::create("nonseq");
            start_item(req);
            req.htrans = ahb_transaction::AHB_NONSEQ;
            req.hburst = ahb_transaction::WRAP4;
            req.hsize  = 3'h2;   // WORD
```

```
            req.hwrite = $urandom_range(0,1);
            req.haddr  = base_addr;
            if (!req.randomize(hwdata)) `uvm_fatal("RAND_FAIL","hwdata rand failed")
            finish_item(req);
            get_response(rsp);

            begin
                bit [31:0] exp = wrap4_addr(base_addr, 0, 2);
                if (rsp.haddr_actual !== exp)
                    `uvm_error("WRAP4",
                        $sformatf("Beat0 addr: exp=0x%08h got=0x%08h",exp,rsp.haddr_actual))
            end

            // SEQ 拍 1~3，逐拍比对地址
            for (int beat = 1; beat < 4; beat++) begin
                req = ahb_transaction::type_id::create($sformatf("seq%0d",beat));
                start_item(req);
                req.htrans = ahb_transaction::AHB_SEQ;
                req.hburst = ahb_transaction::WRAP4;
                req.hsize  = 3'h2;
                req.hwrite = req.hwrite;
                req.haddr  = wrap4_addr(base_addr, beat, 2);
                if (!req.randomize(hwdata)) `uvm_fatal("RAND_FAIL","hwdata rand failed")
                finish_item(req);
                get_response(rsp);

                begin
                    bit [31:0] exp = wrap4_addr(base_addr, beat, 2);
                    if (rsp.haddr_actual !== exp)
                        `uvm_error("WRAP4",
                            $sformatf("Beat%0d addr: exp=0x%08h got=0x%08h",
                                beat,exp,rsp.haddr_actual))
                    else
                        `uvm_info("WRAP4",
                            $sformatf("Beat%0d addr=0x%08h OK",beat,exp),UVM_HIGH)
                end
            end
        end
    endtask
endclass
```

题目 7　SPI CPOL/CPHA 四模式全覆盖 + 写后读校验

难度：中级
标签：SPI / CPOL / CPHA / Mode 0~3 / write-read-back / 协议覆盖

【题目背景】

SPI（Serial Peripheral Interface）协议有四种工作模式， 由时钟极性（CPOL）和时钟相位（CPHA）的组合决定： Mode 0：CPOL=0，CPHA=0（空闲低，上升沿采样） Mode 1：CPOL=0，CPHA=1（空闲低，下降沿采样） Mode 2：CPOL=1，CPHA=0（空闲高，下降沿采样） Mode 3：CPOL=1，CPHA=1（空闲高，上升沿采样） DUT 的 SPI 控制器需支持全部四种模式。验证目标： 在每种模式下写入同一寄存器，立即回读，数据必须完全一致。

【完整要求】

{2'b00, 2'b01, 2'b10, 2'b11}； Step A：发送 SPI_WRITE，随机 wdata，指定 cpol/cpha； Step B：发送 SPI_READ，相同 reg_addr 和 cpol/cpha；

- 1. 定义 spi_modes[4] 数组，枚举四种 {CPOL,CPHA} 组合：
- 2. 对 test_addr=0x1A 依次在四种模式下各执行一次：
- 3. 比对 rsp.rdata 与 wr_data，不一致则 `uvm_error 报告模式编号；
- 4. 成功则打印 `uvm_info 记录模式编号和数据。

【实现要点】

> - cpol/cpha 不随机化，由序列层手动赋值，确保 Write/Read 使用相同模式；
> - 本题是"定向覆盖"题型：foreach 遍历而非 randomize，确保每种模式都被测到；
> - 四种模式的约束：req.cpol = spi_modes[i][1]; req.cpha = spi_modes[i][0]。

【完整答案代码（含全部类定义）】

```
class spi_transaction extends uvm_sequence_item;
    typedef enum bit { SPI_WRITE, SPI_READ } spi_dir_e;

    rand spi_dir_e dir;
    rand bit [7:0] reg_addr;
    rand bit [7:0] wdata;
    rand bit       cpol;
    rand bit       cpha;
    bit      [7:0] rdata;

    `uvm_object_utils_begin(spi_transaction)
        `uvm_field_enum(spi_dir_e, dir, UVM_ALL_ON)
        `uvm_field_int(reg_addr,        UVM_ALL_ON)
        `uvm_field_int(wdata,           UVM_ALL_ON)
        `uvm_field_int(cpol,            UVM_ALL_ON)
        `uvm_field_int(cpha,            UVM_ALL_ON)
        `uvm_field_int(rdata,           UVM_ALL_ON)
    `uvm_object_utils_end

    function new(string name = "spi_transaction");
        super.new(name);
    endfunction
endclass

class spi_mode_coverage_sequence extends uvm_sequence#(spi_transaction);
    `uvm_object_utils(spi_mode_coverage_sequence)

    function new(string name = "spi_mode_coverage_sequence");
        super.new(name);
    endfunction

    task body();
        spi_transaction req, rsp;
        bit [1:0] spi_modes[4] = '{2'b00, 2'b01, 2'b10, 2'b11};
        bit [7:0] test_addr    = 8'h1A;
        bit [7:0] wr_data;

        // 考点：遍历四种 SPI 模式
        foreach (spi_modes[i]) begin
            bit cpol_val = spi_modes[i][1];
            bit cpha_val = spi_modes[i][0];
            wr_data = $urandom();

            `uvm_info("SPI_SEQ",
                $sformatf("=== SPI Mode%0d CPOL=%0b CPHA=%0b ===",i,cpol_val,cpha_val),
                UVM_LOW)

            // WRITE
            req = spi_transaction::type_id::create("spi_wr");
            start_item(req);
            req.dir      = spi_transaction::SPI_WRITE;
            req.reg_addr = test_addr;
            req.wdata    = wr_data;
            req.cpol     = cpol_val;
            req.cpha     = cpha_val;
            finish_item(req);
            get_response(rsp);

            // READ-BACK（同模式回读）
            req = spi_transaction::type_id::create("spi_rd");
            start_item(req);
            req.dir      = spi_transaction::SPI_READ;
            req.reg_addr = test_addr;
            req.cpol     = cpol_val;
            req.cpha     = cpha_val;
            finish_item(req);
            get_response(rsp);

            if (rsp.rdata !== wr_data)
                `uvm_error("SPI_SEQ",
                    $sformatf("Mode%0d mismatch: wrote=0x%02h read=0x%02h",
                        i, wr_data, rsp.rdata))
```

```
            else
                `uvm_info("SPI_SEQ",
                    $sformatf("Mode%0d OK: data=0x%02h",i,wr_data), UVM_MEDIUM)
        end
    endtask
endclass
```

题目 8　Virtual Sequence 串行配置 + 并行数据流同步屏障

难度：高级
标签：virtual sequence / fork/join / sync barrier / p_sequencer / APB / AXI

【题目背景】

实际 SoC 验证中，DUT 在接收数据包之前必须先完成寄存器配置。 典型场景：先通过 APB 总线配置 DUT 的控制寄存器（基地址、数据长度等）， 配置完成后再通过 AXI 总线并行发送两路数据流。 Virtual Sequence 用于协调多个 Agent 的激励顺序。

【完整要求】

任一写失败（pslverr==1）则 `uvm_fatal；

- 1. 实现 virtual_sequencer，包含 apb_seqr 和 axi_seqr 两个句柄；
- 2. 实现 apb_config_sequence，依次写入 3 个配置寄存器，
- 3. 实现 synced_virtual_sequence，用 `uvm_declare_p_sequencer 声明；
- 4. body() 中先调用 cfg_seq.start(apb_seqr) 完成配置（阻塞，串行）；
- 5. 配置完成后用 fork...join 并行启动 axi_seq0 和 axi_seq1；
- 6. 等待两路 AXI 流全部完成后打印完成信息。

【实现要点】

> - 关键错误：用 fork/join_none 启动配置序列——配置还未完成就开始发包；
> - 正确做法：配置序列必须用 join（阻塞等待），只有数据流才用 join；
> - `uvm_declare_p_sequencer 宏让 body() 内可通过 p_sequencer 访问句柄。

【完整答案代码（含全部类定义）】

```
class virtual_sequencer extends uvm_sequencer;
    apb_sequencer apb_seqr;
    axi_sequencer axi_seqr;
    `uvm_component_utils(virtual_sequencer)
    function new(string name, uvm_component parent);
        super.new(name, parent);
    endfunction
endclass

class apb_config_sequence extends uvm_sequence#(apb_transaction);
    `uvm_object_utils(apb_config_sequence)
    function new(string name = "apb_config_sequence");
        super.new(name);
    endfunction
    task body();
        apb_transaction req, rsp;
        bit [31:0] cfg_addrs[3] = '{32'h0,32'h4,32'h8};
        bit [31:0] cfg_vals [3] = '{32'h1,32'h1000_0000,32'h100};
        foreach (cfg_addrs[i]) begin
            req = apb_transaction::type_id::create("cfg");
            start_item(req);
            req.dir    = apb_transaction::APB_WRITE;
            req.paddr  = cfg_addrs[i];
            req.pwdata = cfg_vals[i];
            finish_item(req);
            get_response(rsp);
            if (rsp.pslverr)
                `uvm_fatal("CFG","Config write failed")
        end
        `uvm_info("CFG","DUT configuration complete",UVM_LOW)
    endtask
endclass

class synced_virtual_sequence extends uvm_sequence;
    `uvm_object_utils(synced_virtual_sequence)
    `uvm_declare_p_sequencer(virtual_sequencer)

    function new(string name = "synced_virtual_sequence");
        super.new(name);
    endfunction

    task body();
        apb_config_sequence cfg_seq;
        axi_random_sequence axi_seq0, axi_seq1;

        cfg_seq  = apb_config_sequence::type_id::create("cfg_seq");
        axi_seq0 = axi_random_sequence::type_id::create("axi_seq0");
        axi_seq1 = axi_random_sequence::type_id::create("axi_seq1");

        // 考点：串行配置完成后才并行发数据（不能 join_none）
        cfg_seq.start(p_sequencer.apb_seqr);
        `uvm_info("VSEQ","Config done, starting AXI flows",UVM_LOW)

        axi_seq0.num_trans = 12;
        axi_seq1.num_trans = 12;
        fork
            axi_seq0.start(p_sequencer.axi_seqr);
            axi_seq1.start(p_sequencer.axi_seqr);
        join
        `uvm_info("VSEQ","All flows complete",UVM_LOW)
    endtask
endclass
```